

=====

DATABASE MANAGEMENT SYSTEM - LAB TERM PROJECT REPORT

Project System: Gaming Tournament Tracker Interface

=====

SUBMITTED TO:

Ms. Nadia Shakir

DBMS Lab Course Instructor

SUBMITTED BY:

Abdullah Akbar

Seat Reg ID: 03-134242-005

Computer Science / Section 4-A

Submission Date: April 28th, 2026

PROJECT PROPOSAL OVERVIEW & CORE SCOPE

PROBLEM ENCOUNTERED:

Competitive gaming and esports events are blowing up right now, but most local tournament hosts and gaming clubs use messy Excel sheets, paper logs, or weirdly disconnected documents to track players, match brackets, and overall team standings.

This causes massive issues where the exact same data gets written down multiple times, columns conflict with one another, and finding simple statistics like a team's total points or a player's true win rate becomes a huge headache.

OUR DEFINED SOLUTION:

We designed a relational SQL database setup that pulls all tournament tracking together into one centralized system. By normalizing our table schemas, we eliminated messy data repetition and made it easy to generate live scoreboards, match summaries, and profile records instantly using standard queries.

PROJECT BOUNDARIES:

Our system handles player registrations, team rosters, game genres, scheduling match events, and logging live match point distributions.

Out of scope: The backend does not process real-time game server API handshakes, embedded live streaming video feeds, or automatic match-making systems.

SYSTEM ARCHITECTURE & DESIGN ENTITIES

Below is the structured layout of our tables, including primary data keys (PK) and relational connection points (FK):

1. Player Table: Tracks individual platform accounts.
 - Column keys: player_id (PK), username, full_name, email, country, rank_tier
2. Team Table: Groups gamers together under a team name.
 - Column keys: team_id (PK), team_name, founded_date, captain_id (FK linking to Player)
3. Team_Player Table: The roster connection tracker.
 - Column keys: player_id (FK), team_id (FK), join_date, roster_role
 - Note: Uses a combined Composite Primary Key (player_id + team_id)
4. Game Table: Profiles titles supported by the platform.
 - Column keys: game_id (PK), game_title, genre, platform, max_team_size
5. Tournament Table: Holds active and historical events.
 - Column keys: tournament_id (PK), tournament_name, game_id (FK), start_date, end_date, prize_pool, status
6. Match Table: Brackets showing scheduled team face-offs.
 - Column keys: match_id (PK), tournament_id (FK), team1_id (FK), team2_id (FK), match_date, venue, winner_team_id (FK)
7. Score Table: Detailed ledger showing match point records.
 - Column keys: score_id (PK), match_id (FK), team_id (FK), points_scored, is_winner

OUR STEP-BY-STEP NORMALIZATION PROCESS

To make this database truly stable, we cleaned it up through three explicit steps to hit 3NF status:

FIRST NORMAL FORM (1NF):

Every single data block must hold exactly one value—no comma lists allowed inside a single cell box. In our initial rough draft, we tried saving a match row like this:

Match ID 1 -> Teams: (Alpha, Blade) -> Scores: (25, 12)

This broke 1NF rules. We fixed this by creating a separate Score table where every team's match performance gets its own clean row entry.

SECOND NORMAL FORM (2NF):

The layout must meet 1NF, and non-key details must depend on the entire primary key. In an early design draft of our Team_Player table, we incorrectly included an event name column. But an event name doesn't change based on individual player IDs. We removed it and placed event details cleanly back into the Tournament table where they belong.

THIRD NORMAL FORM (3NF):

The layout must meet 2NF, and columns cannot rely transitively on other non-key columns. We originally considered putting game genre info right inside our Tournament details table.

However, genre changes based on the Game being played, not the Tournament itself. We moved genre fields out to the Game table, using a clean foreign key to pull the data via queries.

REAL DEVELOPMENT ROADBLOCKS & REFLECTIONS

1. Double Table Pointer Issues:

Inside our Match table, we had to reference our Team table twice in the same row—once for Team 1 and once for Team 2. When writing our SQL queries, this initially caused errors. We learned how to fix this by using distinct table aliases (like t1 and t2) so SQL knows exactly which team pointer we are referencing.

2. Enforcing Data Safety:

We configured our database with foreign key safety settings (like ON DELETE CASCADE). This ensures that if a tournament gets wiped from the system, all associated matches and scores clean themselves up automatically, keeping our tables completely free of broken, orphan records.